Learning-Based Control for Automated Perpendicular Parking in CARLA environment[1]

Alkesh K. Srivastava[a]

[a] *University of Maryland, College Park, 20742, Maryland, USA*

Abstract

The existing automatic parking systems deploy a path planner that plans a path to park a vehicle in a particular parking slot. Such path planners subsequently require a path tracking module to track the planned path, but the vehicle's non-linear dynamic causes the vehicle to deviate from its expected behavior. This paper investigates the use of learning-based control to avoid using a path-tracking module and discusses how a reinforcement learning paradigm could result in emergent intelligent behavior. The work discussed in the paper uses a Deep-Q Neural Network and the study conducted uses the CARLA (CAR Learning to Act) simulation environment (Dosovitskiy et al., 2017) for the experiments. Two different methodologies — a classification model with MPC controller and a Regression Model with PID Controller — were studied and compared for the perpendicular parking scenario. The promising result in the CARLA simulation environment demonstrates the applicability of this end-to-end reinforcement learning paradigm in the real world.

Keywords: autonomous driving, automated parking system, deep reinforcement learning, trajectory planning, path-tracking module, end-to-end learning, CARLA, MPC Controller, PID Controller

1. Introduction

The start of the 21st century saw an inclination towards autonomous systems in both industry and research. The DARPA Grand Challenge is a

[1] This work is a final submission for the independent study conducted by Alkesh Kumar Srivastava (Graduate Student, University of Maryland) under supervision of Dr. Nikhil Chopra (Professor, Department of Mechanical Engineering, University of Maryland).

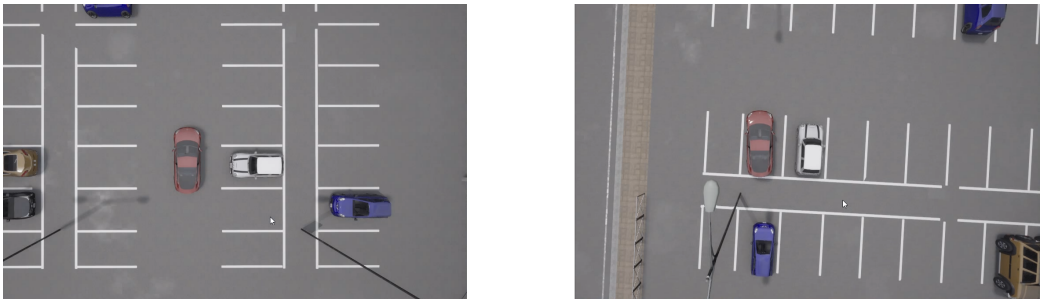


Figure 1: **(LEFT)** - Depicts the starting position of the ego vehicle in CARLA. **(RIGHT)** - Depicts the final position of the ego vehicle facing the pathway of the starting position.

single indication of the enthusiasm shared by the researchers in the Artificial Intelligence community. Various sensors like LiDAR, RADAR, ultrasonic, and cameras were used to make autonomous navigation more robust and reliable. Recent development in the use of the convolutional neural networks in perception using cameras has resulted in groundbreaking performance with just the use of RGB cameras. Currently, Tesla only uses cameras for their autonomous navigation system.

Driving is a complex task. Even for expert human drivers, the complex maneuvers involved in parking a car in narrow spaces prove tedious. While it has become easy for an autonomous vehicle to cruise in a highway setting, it is equally challenging for an autonomous system to park successfully in a narrow space. Perpendicular parking is a complex task; it involves a reversal in the direction of the motion of the ego vehicle and the execution of sharp turns to enter the parking slot. In this paper, we will study the use of a Deep Q-Learning Neural Network to execute the complex maneuver of perpendicular parking. Here, the DQNN model acts as a controller to park the vehicle in a perpendicular parking slot. After the vehicle is parked, the front of the vehicle faces the side on which it began the parking process, as described in Figure 1.

In this study, the vehicle is examined in a simulation environment called CARLA (CAR Learning to Act). It is an open-source simulator developed jointly by Intel Labs, Toyota Research Institute, and Computer Vision Center with the aim to democratize the research development of autonomous urban driving systems. This simulation environment was specifically chosen

because of the flexible options available for not only vehicles but also for various sensors and environmental settings. The CARLA simulator has been tested for end-to-end learning and therefore was the preferred option for the experimental study we aimed to conduct. The only difficulty faced in this CARLA environment was that this environment is designed specifically for driving on roads and not in off-road scenarios. The fact that CARLA uses an internal feature called waypoints for path planning was a hindrance to the study. This was so because the parking lots (or other off-road elements) were not assigned any waypoints and CARLA does not allow the implementation of custom waypoints. The solution to this problem is discussed in Section 3. Thus, CARLA provided a suitable testbed for the parking environment pertinent to the study and the realistic nature of this simulation environment suggests that the promising results of this study could be translated to the real world easily.

This paper proposed the use of Learning-Based Control for automated perpendicular parking. The current automated parking systems deploy a path planner that plans a path to park a vehicle in a particular parking slot. Then, a path tracking module tracks errors in the planned path and attempts to correct them to execute the parking maneuver. The non-linear dynamics of the vehicle cause the vehicle to deviate from the expected behavior. Therefore, we propose an end-to-end reinforcement learning model that uses a convolutional DQN network that eliminates the use of a path-tracking module. The implementation of such a model has also resulted in intelligent behaviors that aid the entire parking process.

The remainder of this work is organized as follows: Section 2 introduces the related work done in the field of automated parking systems. We will discuss both types of contributions that use and do not use Deep Learning Networks for parking. Section 3 will formalize the problem definition. The methodology will be discussed in Section 4. Section 5 will discuss the experiments and their results while Section 6 will draw a conclusion to the study and some remarks on future work.

2. Related Work

In the last two decades, extensive work has been done to explore methodologies to perfect automated parking systems. A two-phase method called probabilistic roadmap (PRM) planning (Kavraki et al., 1996) was used to solve robot motion in static workspaces. However, such a traditional ap-

proach fails when the environment gets dynamic, resulting in an unnecessary collision. Two algorithms called Circle-Circle and Circle-line-Circle (Hsieh and Ozguner, 2008) were also proposed, but they result in a discontinuous path curvature resulting in the need for fitting algorithms. Then Liang et al. (2012) proposed a planning algorithm based on bezier curve fitting to account for continuity. Although it successfully solves the problem of discontinuity, it still does not take care of changing environments. Currently, many automated parking systems use an error tracking controller to aid the path planner. Closed-loop feedback control, Proportional-Integral-Differential (PID) control (Mvemba et al., 2018), and Sliding Mode Control (SMC) (Wu et al., 2019) algorithms have been used to improve the error tracking, but the deviation remains inevitable due to the non-linear dynamic of the vehicle.

Significant work has been done to use learning algorithms in various domains. The advancement in reinforcement learning, specifically in DQN (Mnih et al., 2013), has resulted in unprecedented performance. We have seen how DQN surpassed human-level performance and beat humans in "Go"(Silver et al., 2016). In the last decade, various studies were conducted to incorporate learning algorithms in automated parking systems. In 2017, an implementation with deep reinforcement learning was proposed, but it took tens of days of continuous training. An Actor-Critic architecture based on end-to-end learning was proposed in 2019 (Zhang et al., 2019), but it lacks the robustness of gear switching. An state-based model was also investigated in 2020 (Du et al., 2020). However, this model assumes a system that can take at least 80 actions to perform the parking maneuver.

This paper proposes using an imitation learning-based DQN model that acts as a backbone for two approaches (PID and MPC) to control the vehicle. This adaptation of learning-based control for automated parking systems is novel to the best of our knowledge. This proposed technique not only solves the problem of a dynamic environment (Kavraki et al., 1996; Hsieh and Ozguner, 2008) by considering real-time environment changes but also results in a robust controller that can switch gears based on requirements, unlike Zhang et al. (2019). The model is able to execute complex maneuver while not considering large action space as was consider in Du et al. (2020).

3. Problem Formulation

The inspiration for this work was mainly from the famous work in apprenticeship learning by Abbeel et al. (2006). While Abbeel et al. (2006)

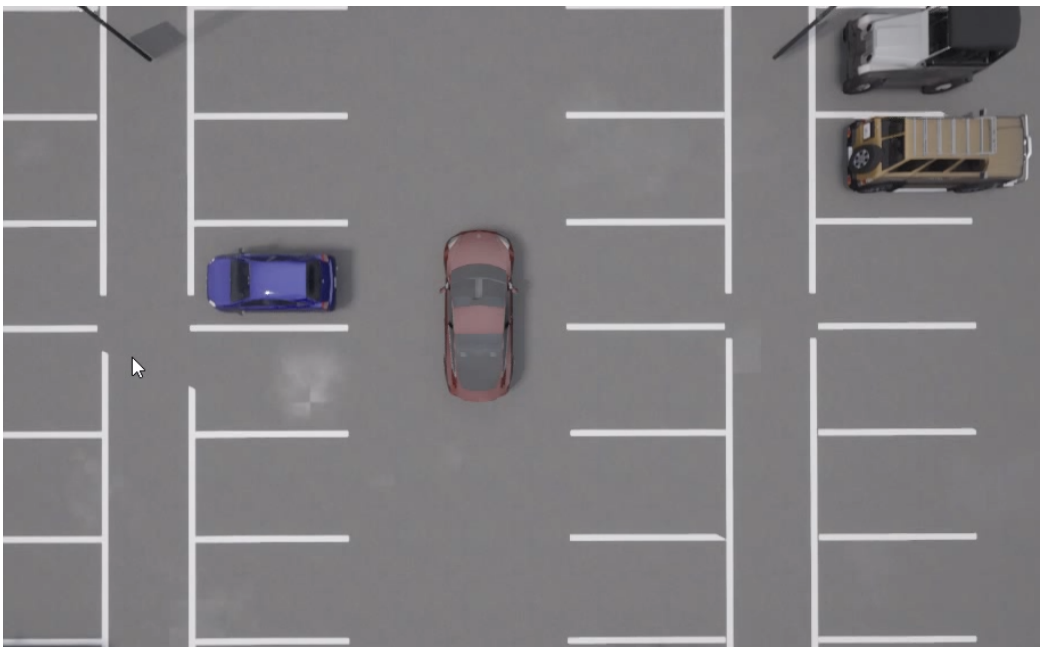


Figure 2: Bird-eye view of the surrounding of the ego vehicle

utilized the estimation of the system's model using a highly optimized extended Kalman Filter and then regression techniques to estimate the coefficient, the work discussed in this paper emphasizes using a convolutional neural network to model the dynamics of the system. In this section, we will formalize the problem statement for the robust regression model, and then we will discuss a relaxed model that performs comparatively well in a controlled environment.

The state of an ego vehicle comprises its position (x, y, z), its orientation (ϕ, θ, ψ), its velocity $(\dot{x}, \dot{y}, \dot{z})$, and angular velocity $(\omega_x, \omega_y, \omega_z)$. As mentioned earlier, the classical modeling techniques use the state-space model, and then estimation techniques are used to evaluate the state-space model coefficient. In this problem, we utilize the visual input of the state through a bird-eye view of the environment surrounding the ego vehicle, as shown in Figure 2.

Figure 3 depicts the convolutional neural network used to train the model. The model is then trained in a supervised fashion using the data generated by an expert-parking planner that regresses to compute the components of the linear velocity vector and the angular velocity vector. The resulting trained

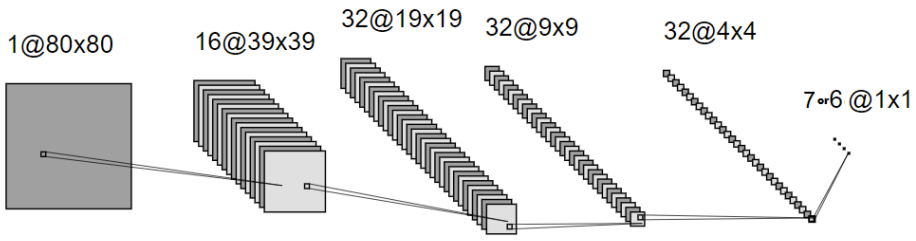


Figure 3: The convolutional neural network used to train the model

model can now be transferred to a Q-Learning network. This reduces the problem to a Markov Decision Process (MDP) to further refine the expertise attained via supervised learning. This MDP problem is solved using the deep Q-Network, a parameterized function of actions and states. Section 4 will discuss the details of the DQN in Section 4.

$$Q(s_t, a_t|\theta) = E[r + \gamma \cdot \max_{a'}(Q(s_{t+1}, a_{t+1}|\theta))|s_t, a_t]$$

The parameter, θ, is approximated by the weights and bias in a deep neural network as shown in Figure 3, updated by minimizing the loss function given below:

$$L(s_t, a_t|\theta_i) = ((r + \gamma \cdot \max_{a'}(Q(s_{t+1}, a_{t+1}|\theta_i))|s_t, a_t) - (Q(s_t, a_t|\theta_i)))^2$$

A relaxed model is also studied in this paper, which uses a classification model instead of a regression model that classifies which action to take instead of regressing over the velocity vectors. The action space on which the model classifies includes THROTTLE_FORWARD, BRAKE, LEFT_STEER_ -FORWARD, RIGHT_STEER_FORWARD, LEFT_STEER_REVERSE, RI -GHT_STEER_REVERSE, and REVERSE. The convolutional neural network shown in Figure 3 is the network used for training the classifier model. Here, the output layer is changed from 6 neurons to 7 neurons. This relaxed model performs comparably with the robust regression model. Section 4 will discuss this relaxed model in detail in Section 4.

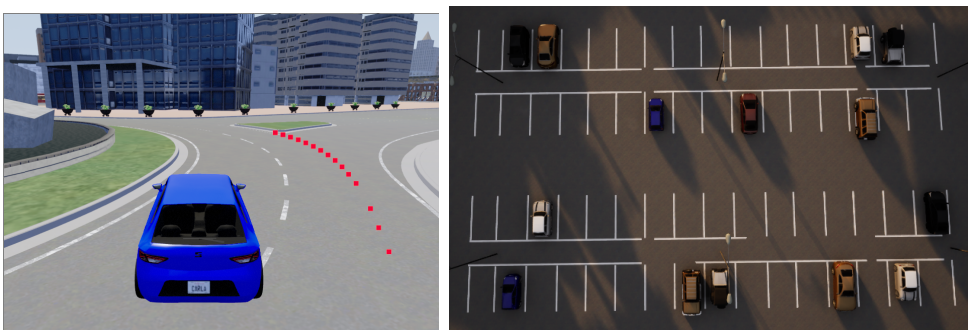


Figure 4: (LEFT) - The red dots represents the waypoints of the vehicle.
(RIGHT) - Snapshot of the parking lot in Town-4.

4. The DQN Approach to Automated Parking - Experimental Setup and Descriptions

In this section, we will be discussing the approach and methodology adopted to design the controller discussed in this paper. As mentioned earlier, the simulation environment we are using is CARLA. CARLA is an open-source simulator that provides a user with an urban setting and valuable assets that we can directly use for autonomous driving. However, CARLA focuses mainly on on-road driving, and it utilizes something called **waypoints** to plan and execute its motion. Waypoints are necessary for any autonomous implementation in this environment, as a waypoint is a point from which the ego vehicle applies control to reach another waypoint. In Figure 4 (LEFT), the red dots are the waypoints the ego-vehicle is supposed to follow.

CARLA has a restriction on creating waypoints. They do not allow custom waypoint generation. CARLA has not mentioned any reasoning behind this in their documentation, but one can speculate that adding unnecessary waypoints on a map may hinder the vehicle's performance. However, our model's requirement is utterly dependent on the off-road elements as we plan to train the model in an off-road setting shown in Figure 4 (RIGHT). Figure 4 (RIGHT) represents a snapshot of a parking lot area in Town04 of CARLA simulator version *0.9.13*. CARLA is a popular open-source simulator, yet there is no publicly available dataset that could be used for the automated parking system. Therefore, the first step in designing the learning-based controller was data generation.

4.1. Data Generation

This paper assumes that the vehicle already knows which parking slot is selected for perpendicular parking. As the CARLA simulator relies upon waypoints, there were two possible options to generate the data for training – (i) manually drive the ego vehicle perfectly to the parking slot and record the data. (ii) create a planner that moves the vehicle without using waypoints. The first attempt in data generation involved (i), but perpendicular parking is a difficult task in real-life, and more difficult is its execution in a computer-simulated environment using a keyboard or video-game controller. After multiple trials, only 30 minutes of useful data was generated using (i). Therefore, it was necessary to create a planner that would produce a control sequence to smoothly move the ego vehicle to a particular parking spot.

A vehicle in the CARLA environment is a 3-dof planar kinematic bicycle model. The front axle is limited to steer within 45 degrees, and the following difference equations describe the motion of the vehicle:

$$X_{t+1} = X_t + V_t \cdot cos(\psi_t + \beta)dt$$

$$Y_{t+1} = X_t + V_t \cdot sin(\psi_t + \beta)dt$$

$$\psi_{t+1} = \psi_t + \frac{V_t}{l_r} \cdot sin(\beta)dt$$

In the above equations, X, Y, ψ are the location and angular rotation of the planar in the CARLA world frame, β is the slip and l_r is the distance between the center of gravity and rear axle.

The real-life experience of reverse perpendicular parking was adopted to create the planner. The vehicle first moves parallel to the assigned parking slot. It aligns the centroid of the bounding box of the ego vehicle with the center of the designated parking slot. It then moves one car length (one bounding box length) forward, reverses, and steers till its body aligns with the parking slot markers and then reverses till successfully parked. The video demonstration of this model can be accessed from this link[2]. Using this algorithm, a total of 4 hours of useful data was generated for the training of two different models. The next subsection will discuss the relaxed model –*Classification Model* – and then discuss the advantages of the more robust –*Regression Model*.

[2]https://youtu.be/q_-mPrZYIpw

4.2. Classification Model

Figure 3 represents the convolutional neural network that is being used to train both the classifier and the regression model. One thing to be mentioned here is that the CARLA simulator has various sensors that could be directly utilized for training. Hence, the model is taking the semantic-segmented image of the bird-eye view as an input. The reason behind this was to prevent the convolutional neural network from learning to segment various objects in the image frame and directly work on state estimation, as described in Section 3.

The only difference in the architecture of the Classification Model and the regression model is the depth of the output layer. This paper refers to the classification model as a relaxed model as it assumes a state-action model of the CARLA environment. In other words, it assumes that the ego vehicle can perform one of the following seven actions at a given time - THROTTLE_FORWARD, BRAKE, LEFT_STEER_FORWARD, RIGHT_STEER_-FORWARD, LEFT_STEER_REVERSE, RIGHT_STEER_REVERSE, and REVERSE. This assumption might seem like a sugar-coated simplification of the complexity of the environment, but in an ideal parking scenario, these are the seven actions that one usually takes. Therefore, this simplification performs equally well as the robust regression model.

For this classification model, Cross-Entropy Loss was used as the loss function, and the model used Adam Optimizer for training, whereas Grid-Search was adopted for hyper-parameter tuning. The result of this model can be seen from the video demonstration linked here. The video demonstration is a piece of evidence that even with a simplified mode, with only seven actions as compared to at least 80 actions of Du et al. (2020), the goal of parking is achieved in a real-world simulator.

One of the significant advantages of using this classification model is that this model can be directly used for end-to-end learning. This model can be made more robust when incorporated with rewards and penalties for different events including, but not limited to, angular orientation with respect to parking lines, collision, and distance between the centroid of the bounding box of the ego vehicle and the center of the parking slot. The model improves with each trial, resulting in a more robust MPC controller. The result of this implementation is currently being investigated and is not included in this paper.

4.3. Regression Model

The regression model is a more robust approach as it intends to predict the linear velocity vector as well as the angular velocity vector estimated from the current state of the environment. This model then utilizes the lateral and longitudinal P.I.D. Controller to guide the vehicle with correct actions to achieve the predicted linear and angular velocity. This results in a more robust controller as it allows the flexibility to make accurate steering angle and accurate throttle pressure resulting in a precise perpendicular parking simulation, unlike the relaxed model. This allows the vehicle to switch gears on the go which was missing in Zhang et al. (2019) For this regression model, Mean Square Error Loss was used as the loss function, and the model used Adam Optimizer for training, whereas Grid-Search was adopted for hyper-parameter tuning.

Although this regression model may seem to be not a fit approach for end-to-end learning, but there is a simple modification to the model that can be transferred this approach to the reinforcement learning paradigm. The resulting output of this regression model gives us the desired vector and leaves the P.I.D Controller on its own. This model, once trained and improved, can then be converted to a model with two actions in the action-space – 1) use Longitudinal P.I.D. Controller 2) use lateral P.I.D. Controller with Longitudinal P.I.D Controller. In general, a vehicle either goes straight or makes appropriate turns. The choice of whether the vehicle is supposed to go straight or is supposed to make a turn is left to the driver. Based on the same idea, one of the two action set is then activated by the convolutional neural network model. In other words, the P.I.D Controller is then activated to make a precise maneuver. This model is then incorporated with rewards and penalties for different events including, but not limited to, angular orientation with respect to parking lines, collision, and distance between the centroid of the bounding box of the ego vehicle and the center of the parking slot. The model improves with each trial, resulting in a more robust P.I.D controller. The result of this implementation is currently being investigated and is not included in this paper.

5. Results

The results in this section was obtained using a computer of the following configuration:

OS Name	Microsoft Windows 11 Home
OS Version	10.0.22000 N/A Build 22000
OS Configuration	Standalone Workstation
OS Build Type	Multiprocessor Free
System Manufacturer	Alienware
System Model	Alienware m15 R3
System Type	x64-based PC
Processor	Intel64 Family 6 Model 165 Stepping 2 Genuine Intel 2592 Mhz
Total Physical Memory	16,146 MB
GPU	NVIDIA GeForce RTX 2060

The following packages are important to run the simulation. The version mentioned alongside is the version of the package used in the experiments:

Package	Version
CARLA	0.9.13
fonttools	4.31.2
h5py	2.10.0
hickle	4.0.4
ipython	7.31.1
jedi	0.18.1
keras	2.8.0
Keras-Preprocessing	1.1.2
libclang	13.0.0
matplotlib	3.5.1
matplotlib-inline	0.1.2
numpy	1.21.5

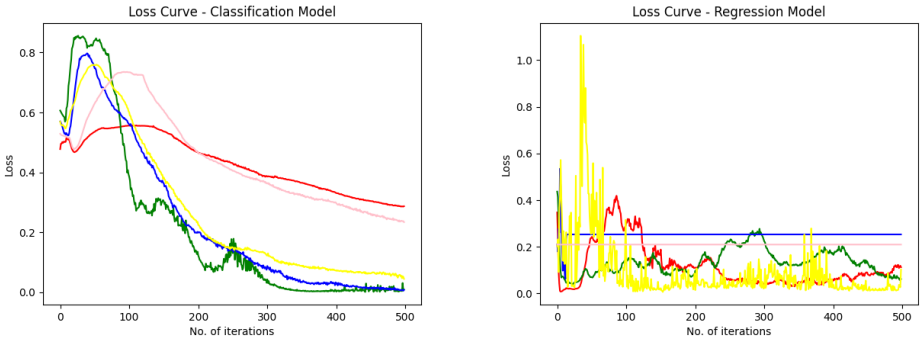


Figure 5: **(LEFT)** - Loss curves of Classification Model.
(RIGHT) - Loss curves of Regression Model.

Package	Version
opencv-python	4.5.5.64
pickleshare	0.7.5
Pillow	9.1.0
pip	21.2.4
prompt-toolkit	3.0.20
pyOpenSSL	22.0.0
setuptools	58.0.4
torch	1.11.0
torchaudio	0.11.0
torchvision	0.12.0
tqdm	4.64.0
urllib3	1.26.9

Figure 5 (LEFT) represents the loss curve obtained during the grid-search hyperparameter tuning for the classification model. It can be very well noted that in some cases, the cross entropy loss is converging to zero, depicting that the classifier model gives accurate results. The output of the classifier model can be accessed from the video link attached here[3].

Figure 5 (RIGHT) represents the loss curve obtained during the grid-search hyperparameter tuning for the regression model. In some cases,the mean square error loss is reaching zero, but then the results are unstable.

[3]https://www.youtube.com/watch?v=HVHxfZMt0VE

This depicts that 4 hours of data is insufficient to generate a robust regression model. Due to limited availability of time in this independent study, exploration of more robust model was not possible. The output of the regression model can be accessed from the video link attached here[4].

The reinforcement model discussed in this paper is currently under investigation and will be included once successfully tested.

6. Conclusion

This paper discussed the possibility and advantage of using learning-based control in the complex task of executing perpendicular parking maneuvers. It discussed how semantic-segmented images could be used to estimate the state of the environment and use it to model the dynamics of the ego vehicle. It examined the results of various models and demonstrated how the model performed in the CARLA simulator environment. One interesting observation in this model was the smart adjustment of vehicles by itself. In some scenarios, the vehicles adjusted their position by throttling initially and then reversing itself to move to the desired parking slot

This paper also discussed transferring the model to an end-to-end reinforcement learning paradigm. This approach was not covered during the short time frame of one semester but is currently being investigated. The results discussed in this paper are promising, and it would be interesting to look forward to the result of the end-to-end reinforcement learning.

References

Abbeel, P., Coates, A., Quigley, M., Ng, A., 2006. An application of reinforcement learning to aerobatic helicopter flight. Advances in neural information processing systems 19.

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V., 2017. CARLA: An open urban driving simulator, in: Proceedings of the 1st Annual Conference on Robot Learning, pp. 1–16.

Du, Z., Miao, Q., Zong, C., 2020. Trajectory planning for automated parking systems using deep reinforcement learning. International Journal of Automotive Technology 21, 881–887.

[4]https://www.youtube.com/watch?v=EQJ_EmRMB3g

Hsieh, M.F., Ozguner, U., 2008. A parking algorithm for an autonomous vehicle. 2008 IEEE Intelligent Vehicles Symposium , 1155–1160.

Kavraki, L., Svestka, P., Latombe, J.C., Overmars, M., 1996. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. IEEE Transactions on Robotics and Automation 12, 566–580. doi:10.1109/70.508439.

Liang, Z., Zheng, G., Li, J., 2012. Automatic parking path optimization based on bezier curve fitting, in: 2012 IEEE International Conference on Automation and Logistics, pp. 583–587. doi:10.1109/ICAL.2012.6308145.

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M., 2013. Playing atari with deep reinforcement learning. URL: https://arxiv.org/abs/1312.5602, doi:10.48550/ARXIV.1312.5602.

Mvemba, P.K., Lay-Ekuakille, A., Kidiamboko, S., Rahman, M.Z.U., 2018. An embedded beamformer for a pid-based trajectory sensing for an autonomous vehicle. Metrology and Measurement Systems 25, 561–575–561–575.

Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al., 2016. Mastering the game of go with deep neural networks and tree search. nature 529, 484–489.

Wu, Y., Wang, L., Zhang, J., Li, F., 2019. Path following control of autonomous ground vehicle based on nonsingular terminal sliding mode and active disturbance rejection control. IEEE Transactions on Vehicular Technology 68, 6379–6390. doi:10.1109/TVT.2019.2916982.

Zhang, P., Xiong, L., Yu, Z., Fang, P., Yan, S., Yao, J., Zhou, Y., 2019. Reinforcement learning-based end-to-end parking for automatic parking system. Sensors 19, 3996.